INF220 / Unidad 2 / Práctica 2

PROGRAMACIÓN ORIENTADA A OBJETOS Y ESTRUCTURAS DE DATOS

Práctica 2 — Recursividad en Java

Carrera	Ingeniería Informática — Facultad de Ciencias Puras, UATF
Asignatura	INF220 · Programación Orientada a Objetos y Estructuras de Datos
Unidad	U2 · Recursividad
Lenguaje / entorno	Java (JDK Temurin 25 LTS) · Visual Studio Code
Modalidad	Trabajo individual
Ponderación	100 pts
Fecha de entrega	12 / 10 / 2026

Ejercicios propuestos

EJ 01 Factorial, potencia y sumatoria

Básico · Recursión lineal · Caso base

Toda función recursiva se apoya en dos piezas: un **caso base** que detiene las llamadas y un **caso recursivo** que reduce el problema a uno más pequeño. Cree la clase `MatematicaRecursiva` con los siguientes **métodos estáticos recursivos**:

DEBE IMPLEMENTAR

- `long factorial(int n)` — calcula n! sabiendo que 0! = 1 y n! = n · (n − 1)!.
- `long potencia(long base, int exp)` — calcula base elevado a exp **sin** usar `Math.pow()`.
- `int sumatoria(int n)` — devuelve la suma 1 + 2 + … + n.
- Lanzar `IllegalArgumentException` si alguno de los métodos recibe un valor negativo.
- Desde `main`, mostrar el factorial de los números del 0 al 20 (límite del tipo `long`) y probar los otros dos métodos con al menos dos valores.

EJEMPLO DE SALIDA ESPERADA

```
0! = 1
5! = 120
20! = 2432902008176640000
2^10 = 1024
Sumatoria(100) = 5050
```

EJ 02 Operaciones con los dígitos de un número

Básico · Recursión sobre enteros

Los dígitos de un entero pueden procesarse recursivamente separando el último con `n % 10` y continuando con el resto del número, `n / 10`. Cree la clase `Digitos` con los métodos:

DEBE IMPLEMENTAR

- `int contarDigitos(int n)` — cantidad de dígitos (considere que 0 tiene un dígito).
- `int sumarDigitos(int n)` — suma de todos sus dígitos.
- `int digitoMayor(int n)` — el dígito de mayor valor.
- `int invertir(int n)` — el número con sus dígitos al revés, apoyándose en un método auxiliar privado `invertir(int n, int acumulado)` que vaya construyendo el resultado.
- `boolean esCapicua(int n)` — indica si el número se lee igual en ambos sentidos, reutilizando `invertir`.
- Probar cada método desde `main` con al menos dos números distintos.

EJEMPLO DE SALIDA ESPERADA

```
Número: 4721
Dígitos: 4   Suma: 14   Mayor: 7
Invertido: 1274   Capicúa: false
Número: 12321  ->  Capicúa: true
```

EJ 03 Cadenas recursivas

Básico · Recursión sobre cadenas

En la Práctica 1 se resolvieron operaciones de texto con bucles. Ahora deberá implementarlas **sin ningún bucle**, reduciendo la cadena en cada llamada mediante `charAt()` y `substring()`. Cree la clase `TextoRecursivo` con los **métodos estáticos recursivos**:

DEBE IMPLEMENTAR

- `String invertir(String s)` — devuelve la cadena al revés.
- `boolean esPalindromo(String s)` — compara el primer y el último carácter y continúa con la subcadena interior, ignorando mayúsculas y espacios.
- `int contarCaracter(String s, char c)` — cuántas veces aparece c en s.
- `int contarVocales(String s)` — cantidad de vocales, sin distinguir mayúsculas.
- `String duplicarCaracteres(String s)` — repite cada carácter dos veces.
- Identificar el caso base de cada método (cadena vacía o de un solo carácter).

EJEMPLO DE SALIDA ESPERADA

```
invertir("recursion")            -> "noisrucer"
esPalindromo("Anita lava la tina") -> true
contarCaracter("banana", 'a')     -> 3
contarVocales("Programacion")      -> 5
duplicarCaracteres("hola")         -> "hhoollaa"
```

EJ 04 Fibonacci y el árbol de llamadas

Básico · Recursión múltiple · Traza

Cuando un método se invoca a sí mismo más de una vez en cada llamada se habla de **recursión múltiple**. La sucesión de Fibonacci (0, 1, 1, 2, 3, 5, 8, …) es el ejemplo clásico. Cree la clase `Fibonacci` con:

DEBE IMPLEMENTAR

- `long fib(int n)` — versión recursiva directa: fib(0) = 0, fib(1) = 1 y fib(n) = fib(n − 1) + fib(n − 2).
- Un atributo estático `contadorLlamadas` que se incremente en cada invocación de `fib` y se reinicie antes de cada prueba.
- `void traza(int n, int nivel)` — imprime cada llamada con una sangría proporcional a su profundidad (puede generarla con `" ".repeat(nivel * 2)`), para visualizar el **árbol de llamadas** de fib(4).
- Mostrar una tabla con n, fib(n) y el número de llamadas para n = 10, 20, 30 y 35.
- Responder en un comentario: ¿por qué crece tan rápido el número de llamadas? ¿Qué subproblemas se repiten?

EJEMPLO DE SALIDA ESPERADA

```
fib(4)
  fib(3)
    fib(2)
      fib(1)
      fib(0)
    fib(1)
  fib(2)
    fib(1)
    fib(0)

n=10   fib=55       llamadas=177
n=20   fib=6765     llamadas=21891
n=30   fib=832040   llamadas=2692537
```

EJ 05 Recorrido recursivo de arreglos

Básico · Arreglos · Recursión con índice

Un arreglo puede recorrerse recursivamente pasando como parámetro el índice de la posición actual. Para que quien use la clase no tenga que conocer ese índice, cada operación se expone mediante un método público que invoca a un **método auxiliar privado** con el parámetro adicional. Cree la clase `ArregloRecursivo` con:

DEBE IMPLEMENTAR

- `int sumar(int[] a)` — suma de los elementos, apoyada en `sumar(int[] a, int i)`.
- `int maximo(int[] a)` — el mayor elemento del arreglo.
- `double promedio(int[] a)` — reutilizando el método `sumar`.
- `int buscar(int[] a, int x)` — búsqueda lineal que devuelve el índice de x o −1 si no existe.
- `void imprimirInverso(int[] a)` — muestra los elementos del último al primero.
- `boolean estaOrdenado(int[] a)` — indica si el arreglo está ordenado de forma ascendente.

EJEMPLO DE SALIDA ESPERADA

```
Arreglo: [7, 3, 9, 1, 5]
Suma: 25   Máx: 9   Promedio: 5.0
buscar(9) -> 2   buscar(4) -> -1
Inverso: 5 1 9 3 7
Ordenado: false
```